

Android设备获取扫码枪扫描的内容与可能遇到的问题解决

主要给大家介绍了关于Android设备获取扫码枪扫描内容的方法，以及在开发中可能会遇到的问题的解决方法，文中通过示例代码介绍的非常详细，对大家的学习或者工作具有一定的参考学习价值，需要的朋友们下面来一起学习学习吧。

前言

大家应该都有所体会，在生活中条形码扫码枪可是随处可见，可以很迅速地扫描出条形码内容，比什么手机相机扫码快了不是一点两点。

为了节约成本，扫码枪可以直接通过蓝牙连接android或其他设备。

那么android设备如何通过蓝牙获取扫描内容的呢？

1. 蓝牙配对，连接设备

打开系统设置，找到蓝牙，打开扫码枪，配对扫码枪设备。输入一个固定的配对码，一般扫码枪说明书里都有写。配对完成后，显示设备已连接。就ok。

2.AndroidManifest中配置权限

android项目中的AndroidManifest.xml文件添加蓝牙权限。

```
<uses-permission android:name="android.permission.BLUETOOTH" />
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />
```

3.检测扫码枪的连接状态

通常来说，扫码枪设备也相当于普通外接输入设备类型，外接键盘。

我这款扫码枪设备返回的是如下蓝牙类型。

BluetoothClass.Device.Major.PERIPHERAL

一般而言，通过如下这种方式就可以获得到我们扫码枪设备的信息。

```
Set<BluetoothDevice> blueDevices = mBluetoothAdapter.getBondedDevices();

if (blueDevices == null || blueDevices.size() <= 0) {
    return false;
}

for (Iterator<BluetoothDevice> iterator = blueDevices.iterator(); iterator.hasNext(); ) {
    BluetoothDevice bluetoothDevice = iterator.next();

    if (bluetoothDevice.getBluetoothClass().getMajorDeviceClass() == BluetoothClass.Device.Major.PERIPHERAL) {
        //TODO 获取扫码枪设备信息
    }
}
```

开发过程中，必然会需要实时判断设备是否正常连接。

```
mBluetoothAdapter.getBondedDevices()
```

这个方法仅仅只能够判断设备是否已配对绑定。但是绑定不代表连接，所以只能放弃。

```
public List getConnectedDevices (int profile)
public int getConnectionState (BluetoothDevice device, int profile)
```

接着又尝试了这两个方法，方法是可用，但是必须要求设备sdk>18,即android 4.3版本以上才可用。

后来转头一想，既然扫码枪也是输入设备，我们可以不同蓝牙设备状态检测入手，改为从输入设备检测入手。于是，

```
private void hasScanGun() {
    Configuration cfg = getResources().getConfiguration();
    return cfg.keyboard != Configuration.KEYBOARD_NOKEYS;
}
```

搞定。

4.获取扫码枪扫描内容

扫描枪，既然是一个外接输入设备，那么很自然的，我们就从KeyEvent入手。

事件解析类

```
/**
 * 扫码枪事件解析类
 */
public class ScanGunKeyEventHelper {

    //延迟500ms，判断扫码是否完成。
    private final static long MESSAGE_DELAY = 500;
    //扫码内容
    private StringBuffer mStringBufferResult = new StringBuffer();
    //大小写区分
    private boolean mCaps;
    private OnScanSuccessListener mOnScanSuccessListener;
    private Handler mHandler = new Handler();

    private final Runnable mScanningFishedRunnable = new Runnable() {
        @Override
        public void run() {
            performScanSuccess();
        }
    };

    //返回扫描结果
    private void performScanSuccess() {
        String barcode = mStringBufferResult.toString();
        if (mOnScanSuccessListener != null)
            mOnScanSuccessListener.onScanSuccess(barcode);
        mStringBufferResult.setLength(0);
    }

    //key事件处理
    public void analysisKeyEvent(KeyEvent event) {

        int keyCode = event.getKeyCode();

        //字母大小写判断
        checkLetterStatus(event);

        if (event.getAction() == KeyEvent.ACTION_DOWN) {

            char aChar = getInputCode(event);

            if (aChar != 0) {
                mStringBufferResult.append(aChar);
            }

            if (keyCode == KeyEvent.KEYCODE_ENTER) {
                //若为回车键，直接返回
                mHandler.removeCallbacks(mScanningFishedRunnable);
                mHandler.post(mScanningFishedRunnable);
            } else {
                //延迟post，若500ms内，有其他事件
                mHandler.removeCallbacks(mScanningFishedRunnable);
                mHandler.postDelayed(mScanningFishedRunnable, MESSAGE_DELAY);
            }

        }

    }

    //检查shift键
    private void checkLetterStatus(KeyEvent event) {
        int keyCode = event.getKeyCode();
        if (keyCode == KeyEvent.KEYCODE_SHIFT_RIGHT || keyCode == KeyEvent.KEYCODE_SHIFT_LEFT) {
            if (event.getAction() == KeyEvent.ACTION_DOWN) {
                //按下shift键，表示大写
                mCaps = true;
            } else {
                //松开shift键，表示小写
                mCaps = false;
            }
        }
    }

    //获取扫描内容
    private char getInputCode(KeyEvent event) {

        int keyCode = event.getKeyCode();
```

```

char aChar;

if (keyCode >= KeyEvent.KEYCODE_A && keyCode <= KeyEvent.KEYCODE_Z) {
    //字母
    aChar = (char) ((mCaps ? 'A' : 'a') + keyCode - KeyEvent.KEYCODE_A);
} else if (keyCode >= KeyEvent.KEYCODE_0 && keyCode <= KeyEvent.KEYCODE_9) {
    //数字
    aChar = (char) ('0' + keyCode - KeyEvent.KEYCODE_0);
} else {
    //其他符号
    switch (keyCode) {
        case KeyEvent.KEYCODE_PERIOD:
            aChar = '.';
            break;
        case KeyEvent.KEYCODE_MINUS:
            aChar = mCaps ? '_' : '-';
            break;
        case KeyEvent.KEYCODE_SLASH:
            aChar = '/';
            break;
        case KeyEvent.KEYCODE_BACKSLASH:
            aChar = mCaps ? '|' : '\\';
            break;
        default:
            aChar = 0;
            break;
    }
}

return aChar;

}

```

```

public interface OnScanSuccessListener {
    public void onScanSuccess(String barcode);
}

public void setOnBarCodeCatchListener(OnScanSuccessListener onScanSuccessListener) {
    mOnScanSuccessListener = onScanSuccessListener;
}

public void onDestroy() {
    mHandler.removeCallbacks(mScanningFishedRunnable);
    mOnScanSuccessListener = null;
}

}

```

Activity中重写dispatchKeyEvent方法，截取Key事件。

```

/**
 * Activity截获按键事件.发给ScanGunKeyEventHelper
 *
 * @param event
 * @return
 */
@Override
public boolean dispatchKeyEvent(KeyEvent event) {

    if (isScanGunEvent(event)) {
        mScanGunKeyEventHelper.analysisKeyEvent(event);
        return true;
    }
    return super.dispatchKeyEvent(event);
}

/**
 * 显示扫描内容
 * @param barcode 条形码
 */
@Override
public void onScanSuccess(String barcode) {
    //TODO 显示扫描内容
}

```

详细代码参看：<https://github.com/czhzero/scangon>

注意点：

1.部分机型无法判断外接键盘信息，如三星。

```
private void hasScanGun() {
    Configuration cfg = getResources().getConfiguration();
    return cfg.keyboard != Configuration.KEYBOARD_NOKEYS;
}
```

三星手机cfg.keyboard返回值等于Configuration.KEYBOARD_NOKEYS。

因此为了更好的兼容，可以采用如下方法，

```
/**
 * 判断是否已经连接扫码枪
 * @return
 */
protected boolean hasScanGun() {

    Set<BluetoothDevice> blueDevices = mBluetoothAdapter.getBondedDevices();

    if (blueDevices == null || blueDevices.size() <= 0) {
        return false;
    }

    for (Iterator<BluetoothDevice> iterator = blueDevices.iterator(); iterator.hasNext(); ) {
        BluetoothDevice bluetoothDevice = iterator.next();

        if (bluetoothDevice.getBluetoothClass().getMajorDeviceClass() == BluetoothClass.Device.Major.PERIPHERAL) {
            return isInputDeviceUsed(bluetoothDevice.getName());
        }
    }

    return false;
}

private boolean isInputDeviceUsed(String deviceName) {

    int[] deviceIds = InputDevice.getDeviceIds();

    for (int id : deviceIds) {
        if (InputDevice.getDevice(id).getName().equals(deviceName)) {
            return true;
        }
    }

    return false;
}
```

Android系统解决扫码枪无法输入字母和字符问题

问题：

在使用扫码枪扫码条码的时候明明有字母和字符，但是输入到Android系统却没哟，输入到电脑是正常的，这就很奇怪，让一个搞上层开发的摸不着头脑，最后和系统讨论才知道是系统按键部分映射被删除导致的。

解决办法：

在Android系统层frameworks/base/data/keyboards文件夹下面有Generic.kl这个文件，此文件为Android默认的按键映射对应表，还有其他的比如：qwerty.kl文件，以及一些自定义码值的kl文件。打开Generic.kl看看类型也许就明白了。

```
key 11 0
key 2 1
key 3 2
key 4 3
key 5 4
key 6 5
key 7 6
key 8 7
key 9 8
key 10 9
key 12 MINUS
key 13 EQUALS
```

key 14 DEL
key 15 TAB

里面是键与键值的映射，比如：键值11 对应的按键为 0 这个，以此类推。那解决就明朗了，将所有字母和字符的按键映射添加进行就ok了，至于按键值是多少我这边直接参考了另外一个平台的Generic.kl文件。重新编译系统验证，此问题解决了。

疑惑问题：

- 用相同Android版本的android.jar 查看keyCode对应的值和Generic.kl文件里描述的不一样，此问题还没有弄明白为什么，系统说两个是不相关的？
- 发现在两个平台上有大部分按键值是一致的，但存在分别的是不一样的，不明白怎么定义的。依据是啥？

总结

以上就是这篇文章的全部内容了，希望本文的内容对大家的学习或者工作具有一定的参考学习价值，如果有疑问大家可以留言交流，谢谢大家对我们的支持。